

UNIVERSIDAD DE LAS PALMAS DE GRAN CANARIA

Escuela de Ingeniería Informática

Práctica 3:
Cifrado Asimétrico

Asignatura: Seguridad de la Información

Curso: 4º de Grado en Ingeniería Informática

Autor:

Nicolás Rey Alonso

`nicolas.rey101@alu.ulpgc.es`

Fecha: Febrero 2026

Índice general

1. Introducción	4
1.1. Objetivo de la práctica	4
1.2. Conceptos fundamentales	4
1.2.1. Criptografía asimétrica	4
1.2.2. Algoritmos utilizados	5
1.2.3. Formatos de claves	5
1.2.4. Esquema híbrido de cifrado	5
1.3. Herramientas utilizadas	6
2. Generación de Claves Asimétricas, Firma y Derivación de Secretos	7
2.1. Claves RSA	7
2.1.1. Descripción teórica	7
2.1.2. Generación del par de claves RSA	7
2.1.3. Exportación de clave pública	8
2.1.4. Conversión a formato DER y de vuelta a PEM	8
2.1.5. Firma y verificación RSA	9
2.2. Claves DSA	9
2.2.1. Descripción teórica	9
2.2.2. Generación de parámetros y claves DSA	10
2.2.3. Conversión a formato DER	10
2.2.4. Firma y verificación DSA	10
2.2.5. Comparación RSA vs DSA	11
2.3. Derivación de secretos: Diffie-Hellman (DH)	11

2.3.1.	Descripción teórica	11
2.3.2.	Generación de claves y derivación	12
2.4.	Derivación de secretos: ECDH con X25519	12
2.4.1.	Descripción teórica	12
2.4.2.	Generación de claves y derivación	13
2.4.3.	Comparación de tamaños: DH vs X25519	13
3.	Intercambio de Información Segura	15
3.1.	Descripción del escenario	15
3.1.1.	Esquema del protocolo	15
3.2.	Generación de claves para Ana y Berto	16
3.3.	Trabajo de Ana: Cifrado y firma	16
3.3.1.	Paso 1: Cifrado del documento	16
3.3.2.	Paso 2: Protección de las claves simétricas	17
3.3.3.	Paso 3: Firma digital	17
3.3.4.	Envío	18
3.4.	Trabajo de Berto: Descifrado y verificación	18
3.4.1.	Paso 1: Recuperación de las claves simétricas	18
3.4.2.	Paso 2: Extracción de clave e IV	18
3.4.3.	Paso 3: Descifrado del documento	19
3.4.4.	Paso 4: Verificación de la firma	19
3.5.	Análisis de seguridad del protocolo	20
3.5.1.	Limitaciones	20
4.	Conclusiones	21
4.1.	Resumen de aprendizajes	21
4.2.	Concclusiones sobre la práctica	21
4.3.	Problemas encontrados y soluciones	22
A.	Comandos OpenSSL Utilizados	24
A.1.	Generación de claves	24

A.2. Firma y verificación	24
A.3. Derivación de secretos	25
A.4. Cifrado asimétrico y conversión BASE64	25

Capítulo 1

Introducción

1.1. Objetivo de la práctica

El objetivo principal de esta práctica es utilizar *OpenSSL* para realizar operaciones criptográficas relativas al cifrado asimétrico, incluyendo:

- Generación y gestión de claves asimétricas (RSA, DSA)
- Exportación y conversión de formatos de claves (PEM, DER)
- Firma digital y verificación de resúmenes
- Derivación de secretos compartidos con Diffie-Hellman (DH) y curvas elípticas (X25519)
- Intercambio seguro de información combinando cifrado simétrico, asimétrico y firma digital

1.2. Conceptos fundamentales

1.2.1. Criptografía asimétrica

A diferencia del cifrado simétrico (donde emisor y receptor comparten la misma clave), la criptografía asimétrica utiliza un **par de claves** matemáticamente relacionadas:

- **Clave pública:** Puede distribuirse libremente. Se usa para cifrar y verificar firmas.
- **Clave privada:** Debe mantenerse en secreto. Se usa para descifrar y firmar.

La relación entre ambas claves cumple las siguientes propiedades:

$$D(K_{priv}, E(K_{pub}, M)) = M \quad (\text{cifrado}) \quad (1.1)$$

$$V(K_{pub}, S(K_{priv}, H(M))) = \text{true} \quad (\text{firma}) \quad (1.2)$$

1.2.2. Algoritmos utilizados

Cuadro 1.1: Algoritmos asimétricos utilizados en esta práctica

Algoritmo	Función	Descripción
RSA	Cifrado y firma	Basado en la factorización de números primos grandes
DSA	Solo firma	Estándar de firma digital del NIST
DH	Intercambio de claves	Protocolo de negociación de secretos compartidos
X25519	Intercambio de claves	DH con curva elíptica Curve25519 (usado por WhatsApp)

1.2.3. Formatos de claves

Cuadro 1.2: Formatos de almacenamiento de claves

Formato	Tipo	Características
PEM	Texto (Base64)	Legible, puede incluir cifrado con contraseña
DER	Binario	Más compacto, no soporta cifrado con contraseña

1.2.4. Esquema híbrido de cifrado

En la práctica real, el cifrado asimétrico no se utiliza directamente sobre documentos grandes debido a su lentitud y sus limitaciones técnicas. En su lugar, se emplea un **esquema híbrido**:

1. Se cifra el documento con un algoritmo simétrico (rápido), por ejemplo AES-256-CBC.
2. Se cifra la clave simétrica utilizada con la clave pública del destinatario (RSA).
3. Se firma un resumen del documento original con la clave privada del emisor.

4. Se envían: documento cifrado + clave cifrada + firma digital.

Con este esquema se logra simultáneamente que solo el destinatario pueda descifrar el documento (confidencialidad), que el documento no haya sido alterado (integridad) y que el emisor sea quien dice ser (autenticación).

1.3. Herramientas utilizadas

- **OpenSSL 3.x:** Herramienta principal de criptografía
- Comandos principales: `genpkey`, `pkey`, `pkeyutl`, `enc`, `dgst`

Capítulo 2

Generación de Claves Asimétricas, Firma y Derivación de Secretos

2.1. Enunciado

3.1 Generación de claves asimétricas (pública-privada), firmado de resúmenes (RSA y DSA) y derivación de secretos con claves DH y ECDH (X25519)

- Generar un par de claves asimétricas RSA de 2048 bits en formato PEM con contraseña.
- Exportar la clave pública a otro fichero PEM.
- Exportar la clave privada a formato DER (binario) y de vuelta a PEM, con otro nombre de fichero. Comprobar que la conversión a formato DER desprotege la clave privada (elimina la contraseña).
- Con el par de claves asimétricas creadas, firmar y comprobar la firma del resumen (con SHA-256) de uno de los ficheros de texto de la práctica anterior.
- Repetir los cuatro pasos anteriores con claves DSA.
- Generar dos pares de claves DH estándar y demostrar que la combinación pública1-privada2 genera el mismo secreto que la combinación privada1-pública2.
- Generar dos pares de claves DH con curva elíptica X25519 (las utilizadas por Whatsapp) y demostrar que la combinación pública1-privada2 genera el mismo secreto que la combinación privada1-pública2.
- Comparar los tamaños de los dos secretos (DH y X25519) generados anteriormente.

2.2. Claves RSA

2.2.1. Descripción teórica

RSA (*Rivest-Shamir-Adleman*) es un algoritmo de criptografía asimétrica basado en la dificultad de factorizar el producto de dos números primos grandes. Permite tanto cifrado como firma digital.

La generación de claves RSA consiste en:

1. Elegir dos primos grandes p y q
 2. Calcular $n = p \cdot q$ (módulo)
 3. Calcular $\phi(n) = (p - 1)(q - 1)$ (función de Euler)
 4. Elegir e coprimo con $\phi(n)$ (exponente público, típicamente 65537)
 5. Calcular $d = e^{-1} \pmod{\phi(n)}$ (exponente privado)
- Clave pública: (n, e)
 - Clave privada: (n, d)

2.2.2. Generación del par de claves RSA

Se genera un par de claves RSA de 2048 bits protegido con contraseña:

Listing 2.1: Generación de claves RSA con contraseña

```
1 # Generar clave privada RSA de 2048 bits (cifrada con AES-256-CBC)
2 openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 \
3   -aes-256-cbc -pass pass:"rsapassword123" \
4   -out rsa_privkey.pem
```

La clave privada generada está cifrada con AES-256-CBC y protegida con la contraseña proporcionada. La cabecera del archivo PEM indica que está cifrada:

Listing 2.2: Cabecera de clave privada cifrada

```
1 -----BEGIN ENCRYPTED PRIVATE KEY-----
```

2.2.3. Exportación de clave pública

Listing 2.3: Exportación de clave pública RSA

```
1 openssl pkey -in rsa_privkey.pem -passin pass:"rsapassword123" \  
2 -pubout -out rsa_pubkey.pem
```

2.2.4. Conversión a formato DER y de vuelta a PEM

El formato DER es la representación binaria de la clave. Una característica importante es que **DER no soporta cifrado con contraseña**:

Listing 2.4: Conversión RSA: PEM → DER → PEM

```
1 # Exportar a DER (se pierde la contraseña)  
2 openssl pkey -in rsa_privkey.pem -passin pass:"rsapassword123" \  
3 -outform DER -out rsa_privkey.der  
4  
5 # Reconvertir DER a PEM (sin contraseña)  
6 openssl pkey -in rsa_privkey.der -inform DER \  
7 -out rsa_privkey_from_der.pem
```

Resultado importante: La clave convertida desde DER tiene la cabecera:

Listing 2.5: Cabecera de clave privada sin cifrar

```
1 -----BEGIN PRIVATE KEY-----
```

En lugar de `-----BEGIN ENCRYPTED PRIVATE KEY-----`, lo que demuestra que la conversión a formato DER **elimina la protección con contraseña** de la clave privada.

2.2.5. Firma y verificación RSA

Se firma el resumen SHA-256 de un fichero de texto utilizando `openssl pkeyutl`:

Listing 2.6: Firma y verificación RSA con SHA-256

```
1 # Crear resumen SHA-256 en binario  
2 openssl dgst -sha256 -binary -out rsa_digest.bin TextFile.txt  
3  
4 # Firmar el resumen con clave privada RSA  
5 openssl pkeyutl -sign -in rsa_digest.bin \  
6
```

```
6 -inkey rsa_privkey.pem -passin pass:"rsapassword123" \  
7 -out rsa_firma.bin  
8  
9 # Verificar la firma con clave pública  
10 openssl pkeyutl -verify -in rsa_digest.bin \  
11 -sigfile rsa_firma.bin \  
12 -pubin -inkey rsa_pubkey.pem
```

El resultado esperado de la verificación es:

Listing 2.7: Resultado de verificación

```
1 Signature Verified Successfully
```

Tamaño de la firma RSA-2048: 256 bytes (igual al tamaño de la clave en bytes: $2048/8 = 256$).

2.3. Claves DSA

2.3.1. Descripción teórica

DSA (*Digital Signature Algorithm*) es un algoritmo de firma digital estandarizado por el NIST. A diferencia de RSA, DSA **solo permite firmar**, no cifrar.

La generación de claves DSA requiere dos pasos:

1. Generación de **parámetros** del dominio (p, q, g)
2. Generación del **par de claves** a partir de los parámetros

2.3.2. Generación de parámetros y claves DSA

Listing 2.8: Generación de claves DSA

```
1 # Paso 1: Generar parámetros DSA  
2 openssl genpkey -genparam -algorithm DSA \  
3 -pkeyopt dsa_paramgen_bits:2048 \  
4 -out dsa_params.pem  
5  
6 # Paso 2: Generar par de claves con los parámetros  
7 openssl genpkey -paramfile dsa_params.pem \  
8 -out dsa_privkey.pem
```

```
8 -aes-256-cbc -pass pass:"dsapassword123" \  
9 -out dsa_privkey.pem  
10  
11 # Exportar clave pública  
12 openssl pkey -in dsa_privkey.pem -passin pass:"dsapassword123" \  
13 -pubout -out dsa_pubkey.pem
```

2.3.3. Conversión a formato DER

Al igual que con RSA, la conversión a DER elimina la protección con contraseña:

Listing 2.9: Conversión DSA a DER y de vuelta

```
1 # Exportar a DER  
2 openssl pkey -in dsa_privkey.pem -passin pass:"dsapassword123" \  
3 -outform DER -out dsa_privkey.der  
4  
5 # DER a PEM (sin contraseña)  
6 openssl pkey -in dsa_privkey.der -inform DER \  
7 -out dsa_privkey_from_der.pem
```

2.3.4. Firma y verificación DSA

Listing 2.10: Firma y verificación DSA

```
1 # Crear resumen y firmar  
2 openssl dgst -sha256 -binary -out dsa_digest.bin TextFile.txt  
3 openssl pkeyutl -sign -in dsa_digest.bin \  
4 -inkey dsa_privkey.pem -passin pass:"dsapassword123" \  
5 -out dsa_firma.bin  
6  
7 # Verificar  
8 openssl pkeyutl -verify -in dsa_digest.bin \  
9 -sigfile dsa_firma.bin \  
10 -pubin -inkey dsa_pubkey.pem
```

Nota: El tamaño de la firma DSA es variable (entre 46-48 bytes típicamente), ya que DSA codifica la firma como estructura ASN.1 con dos valores (r, s) .

2.3.5. Comparación RSA vs DSA

Cuadro 2.1: Comparación entre RSA y DSA

Característica	RSA	DSA
Cifrado	Sí	No
Firma digital	Sí	Sí
Generación de claves	Un solo paso	Dos pasos (parámetros + claves)
Tamaño de firma (2048 bits)	256 bytes (fijo)	~46-48 bytes (variable)
Velocidad de firma	Más lenta	Más rápida
Velocidad de verificación	Más rápida	Más lenta

2.4. Derivación de secretos: Diffie-Hellman (DH)

2.4.1. Descripción teórica

El protocolo **Diffie-Hellman** permite a dos partes derivar un **secreto compartido** sin necesidad de transmitirlo. El principio se basa en la propiedad:

$$(g^a \bmod p)^b \bmod p = (g^b \bmod p)^a \bmod p = g^{ab} \bmod p \quad (2.1)$$

Donde:

- g y p son parámetros públicos
- a es la clave privada del usuario 1
- b es la clave privada del usuario 2
- $g^a \bmod p$ y $g^b \bmod p$ son las claves públicas

2.4.2. Generación de claves y derivación

Listing 2.11: Derivación de secretos DH

```

1 # Generar parámetros DH compartidos
2 openssl genpkey -genparam -algorithm DH \
3   -pkeyopt dh_paramgen_prime_len:2048 \
4   -out dh_params.pem

```

```

5
6 # Generar par de claves #1
7 openssl genpkey -paramfile dh_params.pem -out dh_privkey1.pem
8 openssl pkey -in dh_privkey1.pem -pubout -out dh_pubkey1.pem
9
10 # Generar par de claves #2 (mismos parámetros)
11 openssl genpkey -paramfile dh_params.pem -out dh_privkey2.pem
12 openssl pkey -in dh_privkey2.pem -pubout -out dh_pubkey2.pem
13
14 # Derivar secreto 1: privada1 + pública2
15 openssl pkeyutl -derive -inkey dh_privkey1.pem \
16     -peerkey dh_pubkey2.pem -out dh_secreto1.bin
17
18 # Derivar secreto 2: privada2 + pública1
19 openssl pkeyutl -derive -inkey dh_privkey2.pem \
20     -peerkey dh_pubkey1.pem -out dh_secreto2.bin

```

Resultado: Ambos secretos son **idénticos**, demostrando la propiedad fundamental de Diffie-Hellman.

2.5. Derivación de secretos: ECDH con X25519

2.5.1. Descripción teórica

X25519 es una implementación del protocolo Diffie-Hellman usando la **curva elíptica Curve25519**, diseñada por Daniel J. Bernstein. Es el algoritmo utilizado por aplicaciones como **WhatsApp**, Signal y TLS 1.3.

La ventaja principal es que ofrece seguridad equivalente a claves RSA/DH mucho más grandes:

Cuadro 2.2: Equivalencia de seguridad entre algoritmos

Seguridad (bits)	RSA/DH (bits)	Curva Elíptica (bits)
128	3072	256 (X25519)
192	7680	384
256	15360	512

2.5.2. Generación de claves y derivación

A diferencia de DH estándar, X25519 no necesita generar parámetros previos:

Listing 2.12: Derivación de secretos X25519

```
1 # Generar par de claves X25519 #1
2 openssl genpkey -algorithm X25519 -out x25519_privkey1.pem
3 openssl pkey -in x25519_privkey1.pem -pubout -out x25519_pubkey1.pem
4
5 # Generar par de claves X25519 #2
6 openssl genpkey -algorithm X25519 -out x25519_privkey2.pem
7 openssl pkey -in x25519_privkey2.pem -pubout -out x25519_pubkey2.pem
8
9 # Derivar secretos
10 openssl pkeyutl -derive -inkey x25519_privkey1.pem \
11     -peerkey x25519_pubkey2.pem -out x25519_secreto1.bin
12
13 openssl pkeyutl -derive -inkey x25519_privkey2.pem \
14     -peerkey x25519_pubkey1.pem -out x25519_secreto2.bin
```

Resultado: Ambos secretos X25519 son **idénticos**, confirmando el intercambio correcto.

2.5.3. Comparación de tamaños: DH vs X25519

Cuadro 2.3: Comparación de tamaños de secretos compartidos

Algoritmo	Tamaño secreto	Bits	Observaciones
DH estándar (2048 bits)	256 bytes	2048	Secreto grande
X25519 (Curve25519)	32 bytes	256	Secreto compacto

Conclusión: X25519 genera un secreto de **32 bytes** (256 bits) frente a los **256 bytes** (2048 bits) de DH estándar, logrando seguridad equivalente con un secreto **8 veces menor**. Esto se debe a que la seguridad en curvas elípticas crece mucho más rápido con el tamaño de la clave que en DH clásico.

Capítulo 3

Intercambio de Información Segura

3.1. Enunciado

3.2 Intercambio de información segura (cifrado asimétrico de clave simétrica y firma)

En este apartado vamos a reproducir en una secuencia de operaciones el intercambio de información segura entre dos agentes utilizando cifrado simétrico, cifrado asimétrico de las claves simétricas y firma digital (cifrado asimétrico de un resumen del documento original).

Para ello, generaremos dos parejas de claves RSA de 2048 bits que serán utilizadas por dos agentes (Ana y Berto) de forma que Ana construirá tres ficheros de texto a partir del fichero de texto original, el primero con el fichero cifrado, el segundo con las claves utilizadas y el tercero con la firma digital.

Así, primero generaremos las dos claves:

- Generar una pareja de claves RSA de 2048 bits en ficheros `anapub.pem` y `anapriv.pem` y otra pareja de claves del mismo tipo `bertopub.pem` y `bertopriv.pem`.
- Proteger ambas claves privadas con contraseña “anak” y “bertok” respectivamente.

Se supone que Ana y Berto han intercambiado sus claves públicas. A continuación, realizaremos el trabajo de Ana:

- Cifrar un fichero de texto (`texto.txt`) de los apartados anteriores con AES-256 en modo CBC, con clave y vector escogidos por el estudiante y sin sal, con salida en formato BASE64 a un fichero llamado `cifrado.txt`.
- Crear un fichero de texto (hexadecimal) con la concatenación de la clave y el vector utilizados, de nombre `claves.hex` y cifrarlo con la clave pública `bertopub.pem`, con salida en formato binario a un fichero `claves.bin`

- Convertir claves.bin a claves.txt en formato BASE64 (mediante `openssl enc -a . . .`)
- Obtener un resumen sha256 en binario del fichero de texto original con nombre resumen.bin y firmarlo (es decir, cifrarlo con la clave privada anapriv.pem -clave anak-), con salida en formato BASE64 a un fichero llamado firma.txt.

En este momento, Ana enviaría los tres ficheros obtenidos cifrado.txt, claves.txt y firma.txt (junto a la meta-información relativa a los algoritmos utilizados, cómo se concatenan clave y vector. . .) a Berto. . . que seremos nosotros mismos. Actuando como Berto, procederemos a:

- Convertir el fichero claves.txt a fichero binario claves2.bin con `openssl enc -a . . .`
- Descifrar el fichero claves2.bin con la clave privada bertopriv.pem (contraseña bertok) y salida a un fichero claves2.hex (debería ser idéntico a claves.hex).
- Extraer de claves2.hex la clave y el vector en formato hexadecimal, siguiendo la meta-información que le envió Ana junto con los tres ficheros).
- Descifrar el fichero cifrado.txt con la clave y el vector obtenidos (de claves2.hex) y salida al fichero mensaje2.txt (que debería ser igual al fichero original mensaje.txt utilizado por Ana).
- Verificar que el fichero firma.txt, convertido a binario y descifrado con la clave pública de Ana (anapub.pem), coincide con un resumen sha256 del fichero mensaje2.txt. Muy importante: asegurarse de que utilizan la opción `-verifyrestore` para recuperar el resumen obtenido por Ana descifrando su firma y compararlo con un resumen obtenido por Berto.

3.2. Descripción del escenario

En este apartado se reproduce un intercambio seguro de información entre dos agentes, **Ana** y **Berto**, utilizando un esquema híbrido que combina:

- **Cifrado simétrico** (AES-256-CBC): Para cifrar el documento (eficiente)
- **Cifrado asimétrico** (RSA): Para proteger las claves simétricas
- **Firma digital** (RSA + SHA-256): Para garantizar integridad y autenticación

3.2.1. Esquema del protocolo

El flujo de comunicación es:

1. Ana y Berto generan sus pares de claves RSA e intercambian sus claves públicas.
2. Ana cifra el documento con AES-256-CBC usando una clave y un IV aleatorios.
3. Ana cifra la clave+IV con la clave pública de Berto (solo Berto puede descifrar).
4. Ana firma un resumen SHA-256 del documento original con su clave privada.
5. Ana envía tres ficheros: `cifrado.txt`, `claves.txt` y `firma.txt`.
6. Berto descifra las claves con su clave privada.
7. Berto descifra el documento con las claves obtenidas.
8. Berto verifica la firma de Ana para confirmar integridad y autenticidad.

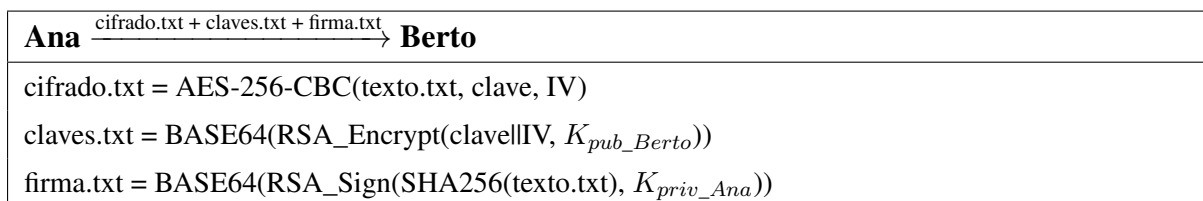


Figura 3.1: Esquema del intercambio seguro Ana $\rightarrow$ Berto

3.3. Generación de claves para Ana y Berto

Se generan dos pares de claves RSA de 2048 bits, protegidos con contraseña:

Listing 3.1: Generación de claves RSA para Ana y Berto

```

1 # Claves de Ana (contraseña: "anak")
2 openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 \
3   -aes-256-cbc -pass pass:"anak" -out anapriv.pem
4 openssl pkey -in anapriv.pem -passin pass:"anak" \
5   -pubout -out anapub.pem
6
7 # Claves de Berto (contraseña: "bertok")
8 openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 \
9   -aes-256-cbc -pass pass:"bertok" -out bertopriv.pem
10 openssl pkey -in bertopriv.pem -passin pass:"bertok" \
11   -pubout -out bertopub.pem

```

Ana y Berto intercambian sus claves públicas (`anapub.pem` y `bertopub.pem`).

3.4. Trabajo de Ana: Cifrado y firma

3.4.1. Paso 1: Cifrado del documento

Ana cifra el fichero `texto.txt` con AES-256-CBC usando una clave y un IV generados aleatoriamente, sin sal y con salida en BASE64:

Listing 3.2: Generación de clave/IV y cifrado del documento

```
1 # Generar clave (32 bytes) e IV (16 bytes) aleatorios
2 ANA_KEY=$(openssl rand -hex 32)
3 ANA_IV=$(openssl rand -hex 16)
4
5 # Cifrar con AES-256-CBC (sin sal, salida BASE64)
6 openssl enc -aes-256-cbc -in texto.txt -out cifrado.txt \
7     -K "$ANA_KEY" -iv "$ANA_IV" -nosalt -a
```

3.4.2. Paso 2: Protección de las claves simétricas

Ana concatena la clave y el IV en un fichero de texto hexadecimal y lo cifra con la clave pública de Berto:

Listing 3.3: Cifrado de claves simétricas con RSA

```
1 # Crear fichero con clave+IV concatenados
2 echo "${ANA_KEY}${ANA_IV}" > claves.hex
3
4 # Cifrar con la clave pública de Berto (salida binaria)
5 openssl pkeyutl -encrypt -in claves.hex \
6     -pubin -inkey bertopub.pem -out claves.bin
7
8 # Convertir a BASE64
9 openssl enc -a -in claves.bin -out claves.txt
```

Formato de `claves.hex`: Los primeros 64 caracteres hexadecimales corresponden a la clave AES-256 (32 bytes) y los siguientes 32 caracteres al IV (16 bytes).

3.4.3. Paso 3: Firma digital

Ana obtiene el resumen SHA-256 del documento original y lo firma con su clave privada:

Listing 3.4: Firma digital del documento

```
1 # Obtener resumen SHA-256 en binario
2 openssl dgst -sha256 -binary -out resumen.bin texto.txt
3
4 # Firmar con clave privada de Ana
5 openssl pkeyutl -sign -in resumen.bin \
6     -inkey anapriv.pem -passin pass:"anak" \
7     -out firma.bin
8
9 # Convertir a BASE64
10 openssl enc -a -in firma.bin -out firma.txt
```

3.4.4. Envío

Ana envía a Berto los tres ficheros junto con la meta-información:

Cuadro 3.1: Ficheros enviados por Ana a Berto

Fichero	Formato	Contenido
cifrado.txt	BASE64	Documento cifrado con AES-256-CBC
claves.txt	BASE64	Clave+IV cifrados con RSA (clave pública de Berto)
firma.txt	BASE64	Firma SHA-256 con RSA (clave privada de Ana)

Meta-información: Algoritmo AES-256-CBC, concatenación clave (64 hex) + IV (32 hex).

3.5. Trabajo de Berto: Descifrado y verificación

3.5.1. Paso 1: Recuperación de las claves simétricas

Berto convierte `claves.txt` de BASE64 a binario y descifra con su clave privada:

Listing 3.5: Recuperación de claves por Berto

```
1 # Convertir BASE64 a binario
2 openssl enc -a -d -in claves.txt -out claves2.bin
3
4 # Descifrar con clave privada de Berto
5 openssl pkeyutl -decrypt -in claves2.bin \
6     -inkey bertopriv.pem -passin pass:"bertok" \
7     -out claves2.hex
```

El fichero `claves2.hex` debe ser idéntico a `claves.hex` de Ana.

3.5.2. Paso 2: Extracción de clave e IV

Siguiendo la meta-información proporcionada por Ana:

Listing 3.6: Extracción de clave e IV

```
1 # Leer contenido
2 CONTENIDO=$(cat claves2.hex | tr -d '\n')
3
4 # Clave AES-256: primeros 64 caracteres hex
5 BERTO_KEY="${CONTENIDO:0:64}"
6
7 # IV: siguientes 32 caracteres hex
8 BERTO_IV="${CONTENIDO:64:32}"
```

3.5.3. Paso 3: Descifrado del documento

Listing 3.7: Descifrado del documento por Berto

```
1 openssl enc -aes-256-cbc -d -in cifrado.txt \
2     -out mensaje2.txt \
3     -K "$BERTO_KEY" -iv "$BERTO_IV" -nosalt -a
```

El fichero `mensaje2.txt` resultante debe ser **idéntico** al `texto.txt` original de Ana.

3.5.4. Paso 4: Verificación de la firma

La verificación se realiza mediante `-verifyrecover`, que permite recuperar el resumen original que Ana firmó:

Listing 3.8: Verificación de la firma digital

```

1 # Convertir firma de BASE64 a binario
2 openssl enc -a -d -in firma.txt -out firma2.bin
3
4 # Recuperar el resumen de Ana (descifrar firma con clave publica)
5 openssl pkeyutl -verifyrecover -in firma2.bin \
6     -pubin -inkey anapub.pem -out resumen_ana.bin
7
8 # Calcular resumen SHA-256 del mensaje descifrado
9 openssl dgst -sha256 -binary -out resumen_berto.bin mensaje2.txt
10
11 # Comparar ambos resúmenes
12 diff resumen_ana.bin resumen_berto.bin

```

Resultado esperado:

- Si `diff` no produce salida: Los resúmenes son **idénticos**.
- **Integridad confirmada:** El documento no ha sido modificado durante la transmisión.
- **Autenticidad confirmada:** Solo Ana (con su clave privada) pudo generar esa firma.

3.6. Análisis de seguridad del protocolo

Cuadro 3.2: Propiedades de seguridad del esquema híbrido

Propiedad	Mecanismo	Justificación
Confidencialidad	AES-256-CBC + RSA	Solo Berto puede descifrar la clave simétrica
Integridad	SHA-256	El resumen detecta cualquier modificación
Autenticación	Firma RSA	Solo Ana posee la clave privada para firmar
No repudio	Firma RSA	Ana no puede negar haber firmado el documento

3.6.1. Limitaciones

- **Sin certificados:** No hay una autoridad de certificación que garantice la identidad. Ana y Berto confían directamente en las claves intercambiadas.
- **Sin protección contra ataques de repetición:** Un atacante podría reenviar los mismos tres ficheros.
- **Sin forward secrecy:** Si la clave privada de Berto se compromete en el futuro, comunicaciones pasadas pueden ser descifradas.

Capítulo 4

Conclusiones

4.1. Resumen de aprendizajes

A través de esta práctica se han consolidado los siguientes conceptos:

1. **Criptografía asimétrica RSA:** Generación de pares de claves, exportación entre formatos (PEM, DER), y la importancia de proteger las claves privadas con contraseña.
2. **Firma digital:** Uso de resúmenes SHA-256 firmados con clave privada para garantizar integridad y autenticación del origen.
3. **DSA:** Algoritmo de firma digital alternativo a RSA, con generación de parámetros independiente de la generación de claves.
4. **Diffie-Hellman:** Protocolo de intercambio de claves que permite derivar un secreto compartido sin transmitirlo directamente.
5. **ECDH con X25519:** Curvas elípticas que logran seguridad equivalente con claves significativamente más pequeñas.
6. **Intercambio seguro:** Combinación de cifrado simétrico (AES-256-CBC), cifrado asimétrico (RSA) y firma digital para lograr confidencialidad, integridad y autenticación.

4.2. Conclusiones sobre la práctica

Es importante tener en cuenta que para el almacenamiento de claves el formato DER no soporta cifrado con contraseña, por lo que se debería evitar su uso para claves privadas a menos que se tomen medidas adicionales de protección.

Para la generación de claves RSA, se recomienda un tamaño mínimo de 2048 bits, aunque 4096 bits es preferible para garantizar seguridad a largo plazo.

En cuanto a la firma digital, es crucial utilizar funciones de resumen seguras como SHA-256 o superiores. Para el intercambio de claves, el uso de esquemas híbridos que combinan cifrado simétrico y asimétrico es esencial para lograr eficiencia sin comprometer la seguridad.

Finalmente, siempre se debe verificar la firma digital antes de confiar en el contenido descifrado para asegurar la autenticidad e integridad del mensaje.

4.3. Problemas encontrados y soluciones

- **Formato DER y contraseñas:** La conversión a DER elimina la protección con contraseña, lo cual hay que tener muy en cuenta al exportar claves privadas.
- **Parámetros DH compartidos:** Ambos pares de claves DH deben usar los mismos parámetros para poder derivar un secreto común, por lo que se genera la duda de cómo asegurar que ambos usuarios usan los mismos parámetros y que estos se mantengan seguros.
- **Tamaño de datos RSA:** RSA solo puede cifrar datos menores al tamaño de la clave (por ejemplo, 256 bytes para RSA-2048), lo que justifica el esquema híbrido.
- **Verificación con `-verifyrecover`:** Es necesario usar esta opción (exclusiva de RSA) para recuperar el resumen original de una firma y compararlo manualmente.

Bibliografía

- [1] OpenSSL Documentation, <https://www.openssl.org/docs/man3.2/>, 2024.
- [2] Rivest, R., Shamir, A., Adleman, L., *A Method for Obtaining Digital Signatures and Public-Key Cryptosystems*, Communications of the ACM, 1978.
- [3] NIST, *Digital Signature Standard (DSS)*, FIPS 186-4, 2013.
- [4] Diffie, W., Hellman, M., *New Directions in Cryptography*, IEEE Transactions on Information Theory, 1976.
- [5] Bernstein, D. J., *Curve25519: New Diffie-Hellman Speed Records*, PKC 2006.
- [6] NIST, *Advanced Encryption Standard (AES)*, FIPS 197, 2001.
- [7] RSA Laboratories, *PKCS #1: RSA Cryptography Specifications*, RFC 8017, 2016.
- [8] Menezes, A., van Oorschot, P., Vanstone, S., *Handbook of Applied Cryptography*, CRC Press, 1996.

Apéndice A

Comandos OpenSSL Utilizados

A.1. Generación de claves

Listing A.1: Comandos para generación de claves

```
1 # RSA: Generar par de claves con contraseña
2 openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 \
3     -aes-256-cbc -pass pass:"contraseña" -out privkey.pem
4
5 # Exportar clave pública
6 openssl pkey -in privkey.pem -passin pass:"contraseña" \
7     -pubout -out pubkey.pem
8
9 # Exportar a DER
10 openssl pkey -in privkey.pem -passin pass:"contraseña" \
11     -outform DER -out privkey.der
12
13 # DER a PEM
14 openssl pkey -in privkey.der -inform DER -out privkey_nopass.pem
```

A.2. Firma y verificación

Listing A.2: Comandos para firma digital

```
1 # Crear resumen SHA-256
2 openssl dgst -sha256 -binary -out digest.bin archivo.txt
3
```

```
4 # Firmar resumen
5 openssl pkeyutl -sign -in digest.bin \
6     -inkey privkey.pem -passin pass:"contraseña" \
7     -out firma.bin
8
9 # Verificar firma
10 openssl pkeyutl -verify -in digest.bin \
11     -sigfile firma.bin -pubin -inkey pubkey.pem
12
13 # Recuperar resumen de la firma (solo RSA)
14 openssl pkeyutl -verifyrecover -in firma.bin \
15     -pubin -inkey pubkey.pem -out resumen_recuperado.bin
```

A.3. Derivación de secretos

Listing A.3: Comandos para derivación DH y X25519

```
1 # DH: Generar parámetros
2 openssl genpkey -genparam -algorithm DH \
3     -pkeyopt dh_paramgen_prime_len:2048 -out dh_params.pem
4
5 # DH: Generar claves con parámetros
6 openssl genpkey -paramfile dh_params.pem -out dh_priv.pem
7 openssl pkey -in dh_priv.pem -pubout -out dh_pub.pem
8
9 # Derivar secreto compartido
10 openssl pkeyutl -derive -inkey priv1.pem \
11     -peerkey pub2.pem -out secreto.bin
12
13 # X25519: Generar claves
14 openssl genpkey -algorithm X25519 -out x25519_priv.pem
15 openssl pkey -in x25519_priv.pem -pubout -out x25519_pub.pem
```

A.4. Cifrado asimétrico y conversión BASE64

Listing A.4: Comandos para cifrado asimétrico

```
1 # Cifrar con clave pública RSA
```

```
2 openssl pkeyutl -encrypt -in datos.txt \  
3     -pubin -inkey pubkey.pem -out datos.bin  
4  
5 # Descifrar con clave privada RSA  
6 openssl pkeyutl -decrypt -in datos.bin \  
7     -inkey privkey.pem -passin pass:"contraseña" \  
8     -out datos_descifrados.txt  
9  
10 # Convertir binario a BASE64  
11 openssl enc -a -in datos.bin -out datos.txt  
12  
13 # Convertir BASE64 a binario  
14 openssl enc -a -d -in datos.txt -out datos.bin
```